

Author: Maiyun Zhang

Affiliation: Pearson College UWC, Victoria, BC
V9C 4H7, Canada

Instructor: Mark Wheen

Title: Monte Carlo method to calculate areas:
a series of attempts to optimize algorithms for a real-world problem

Date of Submission: September 5, 2022

Monte Carlo method to calculate areas: a series of attempts to optimize algorithms for a real-world problem

Maiyun Zhang

September 5, 2022

Abstract

This essay presents a Monte Carlo-based method to calculate the area of a shape and several attempts to refine and optimize the algorithm. Through analyses based on the nature of the computational hardware and the algorithms, we are able to reduce the run time of the naïve method by more than 80 percent while still converging to the correct result. the nature of the computational hardware.

Keywords: Monte Carlo, optimization, integration

Contents

1	Introduction	1
2	Equipments	1
3	General Algorithm	1
4	Process	2
4.1	Initial Approach	2
4.2	Evaluation of the Initial Approach	3
4.3	Scaling Factor	3
4.4	Optimization — Constant Factoring	3
4.5	Optimization — New Algorithm	3
4.6	Optimization — Removing Square Roots	5
5	Conclusion and Evaluation	5
6	Acknowledgement	5

1 Introduction

Monte Carlo methods use random sampling within a given sample space to numerically approximate the real result. With a proper random sampling method, they can approach the real result as the number of trials is increased. Therefore, they are often used on problems when it is non-trivial or even impossible to figure out the solution in closed form. Those methods have a wide range of applications from a computational simulation of complex systems to the calculation of the integral of a simple function.

Finding the area between curves is a traditional mathematical problem with varying degrees of difficulty from being elementary to being impossible. One question is formulated as such: A square has an inscribed circle of radius r . Another circle's center is the lower right vertex of the square, and its radius is $2r$. The two circles intersect at two points. What is the smaller area between the two circles (shown in Figure 1, the specified area is in gray) when $r = 5$?

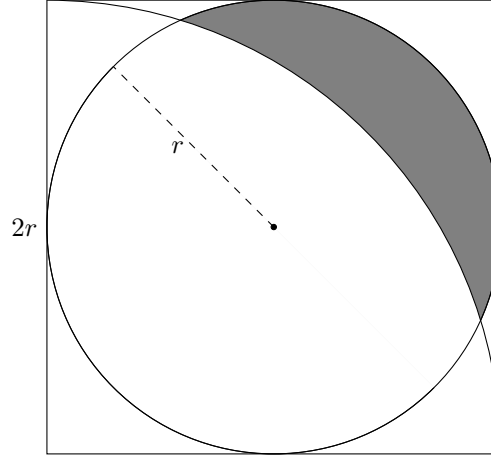


Figure 1: Diagram of the question

It is possible to use trigonometry to work out the area mathematically as shown in (1), which evaluates to approximately $S_{\text{correct}} = 14.638126$ with $r = 5$. However, in this essay, we will examine a way to find the answer with Monte Carlo methods and optimize the method.

$$\begin{aligned} \frac{S}{r^2} &= \pi - \arccos \frac{-\sqrt{2}}{4} - 4 \arccos \frac{5\sqrt{2}}{8} + 2 \sin \left(2 \arccos \frac{5\sqrt{2}}{8} \right) - \frac{1}{2} \sin \left(2\pi - 2 \arccos \frac{-\sqrt{2}}{4} \right) \\ &= \pi - \arccos \frac{-\sqrt{2}}{4} - 4 \arccos \frac{5\sqrt{2}}{8} + \frac{\sqrt{7}}{2} \end{aligned} \quad (1)$$

2 Equipments

All the computations mentioned in this essay are completed on a MacBook Pro with an M1 chip running macOS 12.5.1. The C source files are compiled with LLVM clang version 14.0.6. The optimization flag is **-O3**.

3 General Algorithm

The overall process of performing this Monte Carlo experiment is as follows:

1. Construct a Cartesian coordinate system: the lower left corner of the square in the question is the origin and the vertical side is the positive y direction.
2. Generate a random point (x, y) where $x, y \in [0, 2r]$.
3. Check if the generated random point lies inside the specified area.

4. Increment the counter of points if that random point is in the specified area.
5. Go back to Step 2 for a given amount of random trials.
6. Calculate the fraction of the random points which are in the specified area, then multiply it with the total area to get the result.

With some pseudocode, the above algorithm can be more precisely described as shown in Algorithm 1.

Algorithm 1 General algorithm

```

1: procedure GETAREA( $r, n_{\text{sample}}$ )
2:    $n_{\text{inside}} \leftarrow 0$  ▷ Number of points inside the area
3:
4:   repeat
5:      $x, y \leftarrow \text{GETRANDPOINT}$ 
6:     if INAREA( $x, y, r$ ) then
7:        $n_{\text{inside}} \leftarrow n_{\text{inside}} + 1$ 
8:     end if
9:   until  $n_{\text{sample}}$  trials are completed
10:  return  $4r^2 \frac{n_{\text{inside}}}{n_{\text{sample}}}$ 
11: end procedure

```

For the purpose of this project, we will only work on implementing and improving Step 3. For Step 2, we use the permuted congruential generator considering its speed and statistical idealness (O'Neill, 2014). While this general method is relatively straightforward, there are various adjustments and optimizations that can be done to obtain an optimal version.

4 Process

4.1 Initial Approach

A rather standard way to compute the given result is by simply writing the standard equation of the two circles and using a Monte Carlo simulation to calculate the definite integral. There are three curves of concern on the graph:

1. The upper half of the small circle:

$$\begin{aligned} (y - r)^2 + (x - r)^2 &= r^2 \\ y_{1u} &= r + \sqrt{2xr - x^2} \end{aligned} \tag{2}$$

2. The lower half of the small circle:

$$\begin{aligned} (y - r)^2 + (x - r)^2 &= r^2 \\ y_{1l} &= r - \sqrt{2xr - x^2} \end{aligned} \tag{3}$$

3. The large arc:

$$\begin{aligned} y^2 + x^2 &= (2r)^2 \\ y_2 &= r + \sqrt{4r^2 - x^2} \end{aligned} \tag{4}$$

Looking at the graph, we can see that there are two parts to be considered: the area on the left is between the large arc and the circle, while the area on the right is between two halves of the small circle. The boundary can be found by solving (3) and (4):

$$\begin{cases} x_b = r \frac{5 + \sqrt{7}}{4}, \\ y_b = r \frac{5 - \sqrt{7}}{4} \end{cases}$$

Therefore, as shown in Algorithm 2, if the random point lies to the left of x_b , we decide whether it is within the boundary by checking if $y_2 \leq x < y_{1u}$. Otherwise, we check if $y_{1l} \leq x < y_{1u}$.

Shown in Algorithm 3 is a sample implementation of Algorithm 1 and 2 in the C programming language. It uses the PCG32 generator to generate a random coordinate $(x, y) \in [0, \text{UINT32_MAX}]^2$.

4.2 Evaluation of the Initial Approach

When $r = 5$ and with 1.28×10^9 random samples, the program gives $n_{\text{inside}} = 154869079$, which means the estimated area is 12.0991. Compared to the correct result as calculated in (1), only the first decimal digit is correct. There is a $\frac{12.0991 - S_{\text{correct}}}{S_{\text{correct}}} = 17\%$ error.

The reason why the error from this approach is large is related to the nature of the hardware. The problem is that although **float** is intended to represent a real number, it is not able to distinguish all possible real numbers because a digital computer is discrete in its means of operation. For example, modern computers use the IEEE754 format to store real numbers. In our coordinate range, $[0, 10]$, there are only approximately $2^{30} \approx 10^9$ different possible floats (“IEEE Standard for Floating-Point Arithmetic,” 2019). Although the random generator can theoretically generate 2^{32} different random integers uniformly, when we downsample the numbers to the given range, some possible sample values are lost. As a result, the accuracy cannot continue to increase after saturating the sample space.

4.3 Scaling Factor

Intuitively, we need to attempt to fully utilize the entire possible range of **float** numbers in order to maximize accuracy. For example, if we define

```
1  const float rscaled = radius * rand_samples;
```

and then, we generate random coordinates in this upscaled range:

```
1  const float rmax = 2 * rscaled + 1;
2  x_dot = rmax * pcg32_rand(&thrd_rngx) / UINT32_MAX;
3  y_dot = rmax * pcg32_rand(&thrd_rngy) / UINT32_MAX;
```

With the introduction of the scaling factor, the accuracy improves dramatically with the same amount of random samples. In this case, $n_{\text{inside}} = 187344347$, which gives an estimated area of 14.6363. Now the error decreases to only $\frac{14.6363 - S_{\text{correct}}}{S_{\text{correct}}} = 0.013\%$.

4.4 Optimization — Constant Factoring

Now that the algorithms produce consistent results, it is of our interest to optimize the code so that it takes less time to complete the simulation. One of the first things we can do is factor out the constants from the **for** loops. Three of the major constants that we identified are the scale for the random number generator and the two boundaries:

```
1  const float scale = ldexp(rmax, -32);
2  const float q5msqrt7 = (5 - sqrt(7)) / 4.0, q5asqrt7 = (5 + sqrt(7)) / 4.0;
3  const float rq5msqrt7 = r * q5msqrt7, rq5asqrt7 = r * q5asqrt7;
```

Thus, in each iteration of the loop body, there is a lower number of floating point operations needed, and it therefore saves time.

4.5 Optimization — New Algorithm

A problem with the current approach is that using the standard equations requires calculating multiple floating point square roots. Although a lot of the modern processors offer **sqr**t as an instruction, it is still one of the most time-consuming operations in the simulation. However, mathematically, it is not trivial to further reduce the number of floating point operations.

To further optimize the program, we need another way to implement Step 3 that checks if a point lies in the specified area. Fortunately, we know that if a point lies in a circle, the distance from the center of the circle to the point must be less than the radius of the circle. Using this property, we propose an improved version of Algorithm 2 in Algorithm 4: if the point is outside the big arc but inside the small circle, count it.

Algorithm 2 Point checking based on standard equation

```
1: procedure INAREA( $x, y, r$ )
2:   if  $x \leq x_b$  and  $y_2 \leq y < y_{1u}$  then
3:     return true
4:   else if  $x > x_b$  and  $y_{1l} \leq y < y_{1u}$  then
5:     return true
6:   end if
7:
8:   return false
9: end procedure
```

Algorithm 3 Point checking based on standard equation — C implementation

```
1 uint64_t monte_carlo(const float radius, const uint64_t rand_samples)
2 {
3     const float rmax = 2 * radius + 1;
4     uint64_t i;
5     uint64_t inside = 0;
6     float x_dot, y_dot;
7     float yu, yl, yt;
8     pcg32_random_t rngx, rngy;
9     pcg32_srand(&rngx, UINT64_C(42), UINT64_C(430));
10    pcg32_srand(&rngy, UINT64_C(42), UINT64_C(431));
11
12    for (i = 0; i < rand_samples; ++i)
13    {
14        x_dot = rmax * pcg32_rand(&rngx) / UINT32_MAX;
15        y_dot = rmax * pcg32_rand(&rngy) / UINT32_MAX;
16        yu = y1_upper(x_dot, radius);
17        yl = y1_lower(x_dot, radius);
18        yt = y2(x_dot, radius);
19        /* left segment */
20        if (radius * (5 - sqrt(7)) / 4.0 <= x_dot && x_dot < radius * (5 + sqrt(7)) / 4.0 &&
21            yt <= y_dot && y_dot < yu)
22            ++inside;
23        /* right segment */
24        else if (radius * (5 + sqrt(7)) / 4.0 <= x_dot && yl <= y_dot && y_dot < yu)
25            ++inside;
26    }
27    return inside;
28 }
29
```

Algorithm 4 Point checking based on the length of the hypotenuse

```
1: procedure INAREA( $x, y, r$ )
2:    $(x_{\text{center}}, y_{\text{center}}) \leftarrow (r, r)$ 
3:   if distance between  $(x, y)$  and  $(0, 0) \geq 2r$  and distance between  $(x, y)$  and  $(x_{\text{center}}, y_{\text{center}}) < r$ 
   then
4:     return true
5:   end if
6:
7:   return false
8: end procedure
```

While the standard C library offers a **hypot** function that calculates the distance between two points, its main aim is to prevent overflow instead of to be fast (*Programming languages — C*, 1999). Therefore, in the program, we manually calculate $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ instead of invoking **hypot**. Now the body of the loop becomes:

```
1  x_dot = pcg32_rand(&thrd_rngx) * scale;
2  y_dot = pcg32_rand(&thrd_rngy) * scale;
3  d1 = sqrt((r - x_dot) * (r - x_dot) + (r - y_dot) * (r - y_dot));
4  d2 = sqrt(x_dot * x_dot + y_dot * y_dot);
5  if (d1 < r && d2 >= 2 * r)
6      ++inside;
```

4.6 Optimization — Removing Square Roots

There is still another piece of optimization to complete: it is now possible to completely remove the use of **sqrt** because in $[0, \infty]$, $f(x) = x^2$ is monotonic. Therefore, we only need to compare d^2 and r^2 and there is no need to take the square root.

```
1  static inline float sqhypot(const float a, const float b)
2  {
3      return fma(a, a, b * b);
4  }
```

And the loop body:

```
1  x_dot = pcg32_rand(&thrd_rngx) * scale;
2  y_dot = pcg32_rand(&thrd_rngy) * scale;
3  sqd1 = sqhypot(r - x_dot, r - y_dot);
4  sqd2 = sqhypot(x_dot, y_dot);
5  if (sqd1 < sqr && sqd2 >= 4 * sqr)
6      ++inside;
```

5 Conclusion and Evaluation

The run time (clock time) of all the programs is summarized in Table 1.

Table 1: Run Time (average of three)

Program	Clock time
Initial Approach	13.72 s
Scaled	13.38 s
Fine Tuned	12.08 s
Initial Hypot	10.33 s
Fine Tuned	2.05 s

It is clear that all the approaches reduced the time required to run this Monte Carlo algorithm. Even with compiler optimizations turned on, some small tweaks can still produce visible improvement. What this result suggests is that for critical pieces of code, it is worth it to improve its efficiency manually.

Simultaneously, the previous adjustments did not produce any significant improvement, while changing the algorithm instantly reduced more run time than all the previous attempts. Finally, an improvement to the most heavily-used routine provided an improvement of about 80%. It suggests that while engineering algorithms, we should not focus on premature optimization, but rather work to improve the algorithm itself.

6 Acknowledgement

May I express my gratitude to my instructor, Mark, who supported my exploration along the way.

Thank my parents for providing me with a perfect environment to learn and explore the world of computing.

References

- IEEE standard for floating-point arithmetic. (2019). *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, 1–84. <https://doi.org/10.1109/IEEESTD.2019.8766229>
- Programming languages — C* (Standard). (1999). International Organization for Standardization. Geneva, CH.
- O’Neill, M. E. (2014). *PCG: A family of simple fast space-efficient statistically good algorithms for random number generation* (tech. rep. No. HMC-CS-2014-0905). Harvey Mudd College. Claremont, CA. <https://www.cs.hmc.edu/tr/hmc-cs-2014-0905.pdf>